

Introduction

For my Backend Development module I decided to create a weather and map application. The idea that I began with was to build something that was both practical and also slightly interactive attempting to get to the point in which my project lets my users search for any location and then get detailed weather data from the API I added along with a live map at the top of the screen of their chosen place. In this I wanted my project to feel like my users could genuinely use this in their every day to day life, one of the reasons in me making this project was because the weather is something in which that I am constantly checking from the Met Office to Apple's weather app however I wanted to create one of my own with more simplistic features and an overall easy to understand layout that gives you all of the information that you need weather you need to plan a trip away, check the weather before heading out wondering if you need a coat or you are just curious about other parts of the world and what their weather is like.

Although it may seem like the front end development of my project plays a big role in how the app looks and feels to navigate the real focus on this project was on using the backend part to this and attempting to create a fully functional weather and map application specifically with API's to get present real time dynamic data. This included pulling the real time weather information, showing forecasts, handling the location input and linking all of this with map visuals that are updated based on the users searches and their current location that is pulled when the program boots up.

Throughout working on this project I learnt a lot about working with external API's and managing the data in VS Code while still managing to structure my code in a way that keeps the data reusable and neat. While I have more than a general understanding of how I wanted things to work from the start, I also needed to do lots of research and experimenting after class lessons to fully grasp the concept and to actually make my project come together. There were many times in this where I obviously didn't immediately know how to implement something with this being my first time implementing an API and its data so in these cases I relied on various resources on the internet such as

["https://www.w3schools.com/js/js_api_intro.asp"](https://www.w3schools.com/js/js_api_intro.asp),

["https://developer.mozilla.org/en-US/docs/Web/API"](https://developer.mozilla.org/en-US/docs/Web/API) and other various resources, these tools were essential in helping me figure things out as I went along using trial and error to make sure that I got things right. While I didn't always understand things instantly, I did use those resources to build up my knowledge of API implementation as I found it very tricky at first and now in the end I am very pleased with how this work has turned out.

Project Goals and Concept

The main goal of my project was to create a web application that combines the real time weather data with an interactive map which would give the users a very easy way to check the current conditions and explore the weather conditions in different locations anywhere across the world. I wanted this project to be clean, simple to use and genuinely helpful as something that would feel more like a useful tool for my users and myself included rather than just a demo that it showed off. This application i made is built around a few of the key ideas i had when beginning this project with one of them being the **users convenience**, as my project should automatically detect the user's current location and show the local weather to the user by gaining their exact location data as soon as the program is loaded so that my application can be more efficient for my users. Another concept i wanted to add was definitely **searchable locations**, i wanted to add searchable locations in this project because i think that it is necessary for my users to be able to look up any city or area that they are interested in, however this was not as easy as it seemed when implemented it began to be a struggle not only implementing the data but also figuring out the CSS styling i had to do lots of research in order to complete this. **Clear and accessible information** was another thing that I thought was key to making my application as professional as it can be, the weather details like temperature, humidity, wind speed and forecast should be displayed clearly and without overwhelming the user. The reason I wanted to take this route was because I find that most weather applications can be very overwhelming with information on their page when in reality I just want to know the temperature on a certain day, place and time with no other excessive information given. Another concept i thought i needed to add was **Visual Context**, the application that i made includes a map that i used from Leaflet.js and its purpose is so that users can actually clearly see where the location is on the map which gives the user more visual context and also gives the user a sense of reassurance that the weather information is correct as my map pinpoints exactly where you are on the map. The last concept i wanted to add was some **User Customisation** this being that the user can have buttons that control features like dark mode and temperature unit conversion which gives my users control over how they interact with the application and also makes my project accessible to more users as i know not everyone knows the temperature in °C so there is a toggle option for the fahrenheit.

The reason in which i chose to make this project with this API compared to the many other exciting and interesting ones i could have created using other API's was firstly because i thought that this project would be the most intuitive to help me learn more about API's and how to implement them with this application being slightly simple and easier to understand than most other API's. However I also chose to do this project genuinely because I wanted something that involved live data, user interaction and an application that would not go out of date and can be used forever with opportunities to upgrade this app and learn/work more with adding potentially more and more API's to this project. Weather apps are a common type of real world tool however combining one with an interactive map also made this more interesting for me and actually pushed me to think about how to coordinate different types of data and user input. It also gave me a good challenge from the backed development perspective even though the code does run in the browser everything depends on API requests, handling the responses and even updating the whole user interface based on whatever the results are.

Basic Goal And How I Propose To Accomplish It

The basic goal of my project that i had in mind was to build a weather and map application that can firstly detect a user's location automatically and show the weather information for that given place, let the users of my application search for any city and then instantly see the updated weather information and data, show the selected location on a live map so that there is a direct visual connection for the user between them and the data given and finally i wanted my application to include some sort of extra features such as a 5 day forecast so that my users can plan ahead of time, the temperature conversion between degrees and fahrenheit and a dark mode for a better user experience. Essentially I wanted to create a small, but also very useful tool that interacts with real time backend data and the API that I chose and dynamically update that data based on what the user does with this app.

How i planned to build it

From the start of this project I knew I'd be working mostly in javascript, HTML and CSS handling the layout and the style of the page. The backend side of things with my project comes in with how I handle and fetch the live data from my chosen API. I broke this problem down using problem decomposition by managing this big challenge that I had and breaking it down into smaller more manageable tasks such as the first one being the user's current geolocation.

Users Current Geolocation

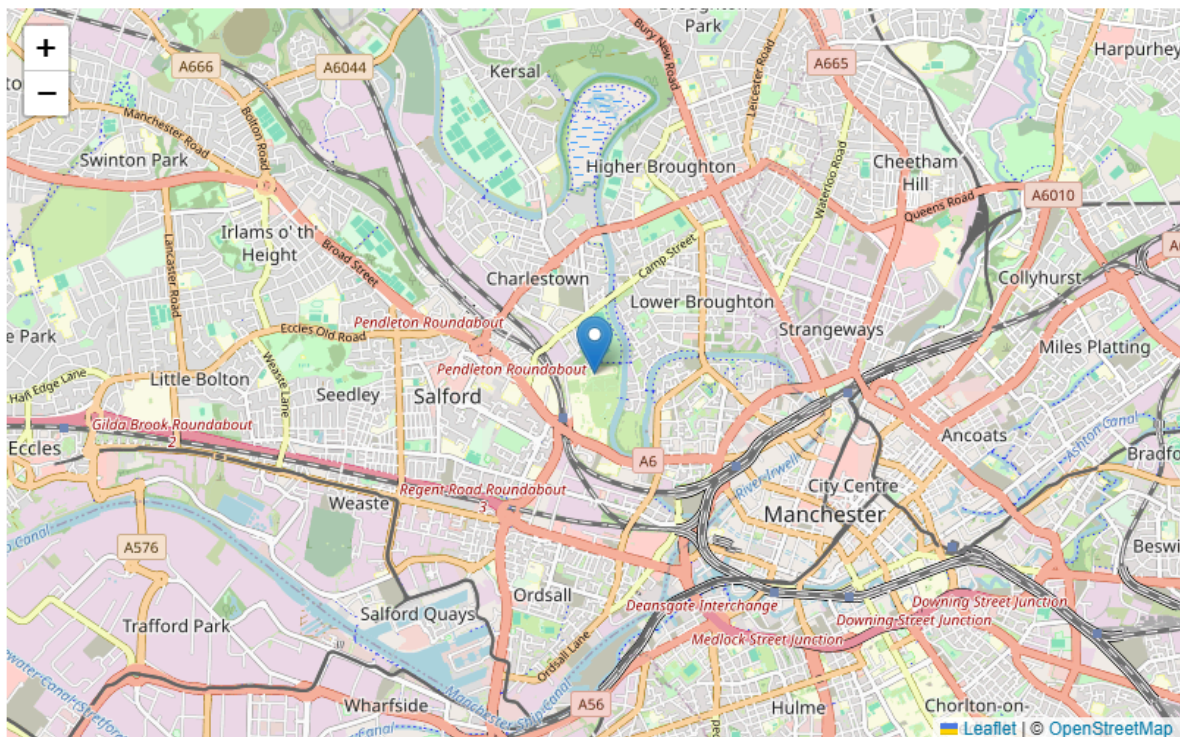
I used the built in Geolocation API in the browser that is provided to get the user's current coordinates. This meant that the application could immediately fetch the weather data as soon as it loaded and as soon as the user pressed the prompt button on their browser to allow my application to gather their coordinates. Doing this means that the app is much more accessible for my users as there is no need to search for the weather information for their current place as my code picks it up for them instantly.

Weather Data Fetching

The **OpenWeatherMap API** that i had chosen was used to get the current weather, this being the temperature, humidity and wind speed etc. It was also used for things like fetching the 5 day forecast and returning the weather icons and their descriptions. This required setting up `fetch()` calls in the javascript part of things and using the API key and building the right URL query parameters like the city name or the latitude and longitude coordinates. This API was more or less the core of my whole project as it provided everything I needed to bring my ideas together and overall really helped in making this project.

Dynamic Map Integration

For this application I chose **Leaflet.js** because it was lightweight and easy to use for my dynamic and interactive maps. I also chose this as I thought it would be the best thing to experiment on when learning how to implement APIs for the first time and also we went through the basic understandings in class on **Leaflet.js**. The map I chose updates based on the user's current location (which updates when the app is loaded) and also the results of the search that is entered by a user. Whenever a new location is detected or searched by the map will update with a clear visual marker and centre on that specific area and you can even enter postcodes into the search bar to help get more precise exact geolocations and that exact place's weather data/information.



Searching Functionality

To make my application and project more flexible I wanted to add an input field where users can type in any city name or postcode. I wanted to do this as I thought that this would be necessary for a working functional and helpful weather and map application, I did want this project page to be simple and sleek however I think that an interactive map is a nice touch while being very necessary for my users and still keeping to my simple concept I wanted to achieve. When my users press Enter or click on the search icon that I created, the app then fetches the new weather data and also re-centers the map according to what location you entered.

Forecast Display

For the forecast, I filtered through the data returned by **OpenWeatherMap's** 5 day forecast endpoint. I figured out that the API that I had chosen gives the results in 3 hour intervals so I picked one data point per day to show a simplified version but also a useful 5 day view. The reason that i did this was because like i have previously mentioned i want the page to be simplistic, one of the things i didn't want in my project was the overwhelming information about every hour of the day and what the temperature id and wind speed etc.

5-Day Forecast

08/05/2025

scattered clouds

Temp: 13.22°C

09/05/2025

scattered clouds

Temp: 15.74°C

10/05/2025

clear sky

Temp: 17.21°C

11/05/2025

few clouds

Temp: 18.19°C

12/05/2025

light rain

Temp: 18.92°C

Dark Mode Toggle

To create the dark mode toggle which was a simple feature that i wanted to add all that was needed was some simple checkbox and CSS variables to build this dark mode feature. When this is enabled the whole app switches to dark colours which i think is better for use at night or in a low light environment. I created this as personally i use low light features on my pc and have features set to turn night light on when it gets dark out, so if my users are anything like me and hate the light being too bright on some applications when its dark there is always the option for a dark mode toggle there.

Toggle Dark Mode

Temperature Unit Switch

With some people preferring Fahrenheit and others preferring to use degrees Celsius i added both into my project so that my users have more functionality over this application. This updates the way that the temperature is displayed so that all of my users are able to use this app.

Toggle °C/°F

Functions And My Process

In this section of my write up I will go through the key parts of my code and explain what they do, how they work and how I went about building them and why. This part of my project I think helped me to break down bigger tasks into smaller more manageable functions especially when dealing with API's and the user interaction. Some parts of the coding process involved using online resources such as OpenWeatherMap API documentation, The leaflet.js documentation and W3 schools, this helped me with some of the syntax for fixing errors that popped up.

Getting The Users Location `initialize()`

One of the first features that I decided to implement was automatically detecting the users location. This is done using the browsers Geolocation API, this is the code i used.

```
async function initialize() {
  if (navigator.geolocation) {
    navigator.geolocation.getCurrentPosition(async (position) => {
      const { latitude, longitude } = position.coords;
      makeChart(latitude, longitude);
      const weatherData = await fetchWeatherData(latitude, longitude);
      displayWeatherData(weatherData);
      displayForecast(weatherData);
    }, (error) => {
      console.error("Geolocation error:", error);
      alert("Failed to get your location. Please enable location services and try again.");
    });
  } else {
    alert("Geolocation is not supported by this browser.");
  }
}
```

The code i used checks if geolocation is available in your browser and if it is then begins to retrieve the coordinates from the browsers API and pass them to two other functions them being the one to get the weather `fetchWeatherData` and then one that updates the map

`makeChart(latitude, longitude);` if the geolocation is not available in the users browser then it shows an alert that being

`alert("Geolocation is not supported by this browser.");` . another thing to add is if geolocation is available in your browser however you have not allowed it the application comes up with this error message

```
alert("Failed to get your location. Please enable location services and try again.");
```

.I think this is a nice way to make the applications feel more responsive and this was done so that the users can quickly and efficiently see their exact weather for the next 5 days and now in real time quickly.

Searching for a location `searchLocation()`

This function lets users search for a place manually if maybe they don't want to use their location or just fancy seeing what the weathers like in the place they want to visit or are just interested.

```
async function searchLocation() {
  const location = document.getElementById("locationInput").value;
  if (!location) {
    alert("Please enter a location.");
    return;
  }

  const geocodingUrl =
`https://nominatim.openstreetmap.org/search?q=${encodeURIComponent(location)}&format=json&limit=1`;

  try {
    const response = await fetch(geocodingUrl);
    const data = await response.json();
    if (data.length === 0) {
      alert("Location not found. Please try again.");
      return;
    }

    const { lat, lon } = data[0];
    makeChart(lat, lon);
    const weatherData = await fetchWeatherData(lat, lon);
    displayWeatherData(weatherData);
    displayForecast(weatherData);
  } catch (error) {
    console.error("Failed to search location:", error);
    alert("Failed to search location. Please try again later.");
  }
}
```

This code uses The OpenStreetMaps API to convert the location's name into coordinates and if there is a valid result found for the city or country or even postcode that my user has entered then the application will fetch and display the new map and weather info accordingly. This is very effective in my application as you can also see at the end there are `alert` and `console.error` cautions in place to make sure my user knows why their input may not be working with the map and weather data.

Displaying the map `makeChart(lat, lon)`

This function `makeChart(lat, lon)` handles everything in my backend related to showing the **Leaflet.js map**. During the development of this project I actually wrote this section of my backend code later on to improve its functionality as there were some errors with this and some things in which I did not like about it. Here is the code below

```

async function makeChart(lat, lon) {
  if (!map) {
    map = L.map("map").setView([lat, lon], 13);
    L.tileLayer("https://tile.openstreetmap.org/{z}/{x}/{y}.png", {
      maxZoom: 19,
      attribution: '&copy; <a
href="http://www.openstreetmap.org/copyright">OpenStreetMap</a>',
    }).addTo(map);
  } else {
    map.setView([lat, lon], 13);
  }

  if (marker) {
    map.removeLayer(marker);
  }
  marker = L.marker([lat, lon]).addTo(map);
}

```

The function here checks if the map has already been made and if that variable comes back as false then it sets the map up; however if the map already exists then it updates the view and location that the user entered or their geolocation and replaces the last one with the existing marker. Originally I thought that the program created a new map every time which caused some errors in other parts of the code however after looking at the **Leaflet.js documentation** I learned how to update the map properly without pressing the refresh button or restarting the application.

Getting weather info from OpenWeatherMap `fetchWeatherData(lat, lon)`

This function here below does the main core part of my backend development as this function does the actual API call to **OpenWeatherMap** using the coordinates and then it returns the current and daily weather data to my code to then compute an output from this.

```

// Function to fetch weather data from the API
async function fetchWeatherData(lat, lon) {
  const url = `${BASE_URL}?lat=${lat}&lon=${lon}&units=metric&appid=${API_KEY}`;
  try {
    const response = await fetch(url);
    if (!response.ok) {
      throw new Error("Network response was not ok");
    }
    return response.json();
  } catch (error) {
    console.error("Failed to fetch weather data:", error);
    alert("Failed to fetch weather data. Please try again later.");
  }
}

```

In this is used the `async` function this is used because it makes sure that the data loads before trying to display it when nothing is there, this is good to help with the loading of my application to the browser. Also any errors that there are, are shown and said with the alert sign which sends a message to the user saying "Failed to fetch weather data. Please try again later".

Showing today's weather `displayWeatherData(data) {`

This Function `displayWeatherData(data) {` extracts all of the useful data from the API response and then update the webpage accordingly shown in the code below

```
// Function to display weather data on the page
function displayWeatherData(data) {
  if (!data) return;
  const description = data.current.weather[0].description;
  const temperature = data.current.temp;
  const humidity = data.current.humidity;
  const windSpeed = data.current.wind_speed;
  const icon = data.current.weather[0].icon;

  document.getElementById("description").innerHTML = `
    
    Cloud formation: ${description}
  `;
  document.getElementById("Temperature").innerText = `
  Temperature: ${temperature}°C`;
  document.getElementById("Humidity").innerText = `Humidity:
${humidity}%`;
  document.getElementById("Wind Speed").innerText = `Wind Speed:
${windSpeed}m/s`;
}
```

This code adds each different icons for a visually pleasing touch for my users so that they can clearly understand what the weather is going to be like or is, and it also displays the temperature, humidity and wind speed these are the three main metrics in which i think are most necessary for my users to know. The wind speed is displayed in miles per second

```
document.getElementById("Wind Speed").innerText = `Wind Speed: ${windSpeed}m/s`;
```

The humidity is displayed in percentage

```
document.getElementById("Humidity").innerText = `Humidity: ${humidity}%`;
```

And the Temperature is displayed in degrees celsius (or fahrenheit)

```
document.getElementById("Temperature").innerText = `Temperature: ${temperature}°C`;
```

5 day forecast implementation `displayForecast(data)`

The 5 day forecast implementation loops through the APIs daily array that is imbedded in their system and then shows the weather for the next 5 days

```
// Function to display a 5-day weather
function displayForecast(data) {
  if (!data || !data.daily) return;

  const forecastContainer =
document.getElementById("forecastContainer");
  forecastContainer.innerHTML = ""; // Clear previous forecast

  data.daily.slice(1, 6).forEach((day) => {
    const date = new Date(day.dt * 1000).toLocaleDateString();
    const temp = day.temp.day;
    const description = day.weather[0].description;

    const forecastItem = document.createElement("div");
    forecastItem.classList.add("forecast-item");
    forecastItem.innerHTML = `
      <p><strong>${date}</strong></p>
      <p>${description}</p>
      <p>Temp: ${temp} °C</p>
    `;
    forecastContainer.appendChild(forecastItem);
  });
}
```

When the `fetchWeatherData(lat, lon);` is called i get a full weather report from the **OpenWeatherMaps API** and this includes in the API the current weather right now, the hourly weather and the daily weather forecast for the next 7 to 8 days however in this feature we are only interested in the daily array. In the line

`data.daily.slice(1, 6).forEach((day) => {` i am using the `.slice(1, 6).` code to skip todays forecast as todays forecast in the array would be set at the Index 0 so thats why we have gone from 1 to 6 to ensure we get the 5 days after. To get this to show presentably on the page each forecast is turned into a small block

```
<!-- 5-Day Forecast -->
<div class="forecast">
  <h2>5-Day Forecast</h2>
  <div id="forecastContainer"></div>
</div>
```

With the date, weather description and the day temperature all in their correct readable format.

Problems i Faced and How i Overcame Them

Like every coder with their projects mine definitely did not go perfectly from the start. I ran into quite a few problems over the development of this application, some of the problems being small but frustrating and some others being more technical but each of these problems helped me learn how to correctly implement APIs overtime.

Map reloading Every time

The problem that i faced was when testing the app i noticed that every time i wanted to search for a new location the map would then reload completely and then sometimes also flicker and reload awkwardly. This was a small but very frustrating error for me as it took me a while to figure out the fix for this. I learnt that this happened because I was creating a new **Leaflet** map instance every time that a new location was searched without checking if one had already existed. To fix this i had to change some things around with the

`makeChart(lat, lon)` function to check if the map had already been created or not.

Issue with search function

The problem i faced here was that if the user would press search without typing up a location then the app would break and stop working while coming up with random unknown errors.

The way that this was fixed was to simply add in an input validation into the

`searchLocation()` function to ensure and check that the input was empty then if it was the user would get a clear alert message pop up on screen to show them why the map isn't location the location that they are typing, this was a good spot from me as i think that the error handling and user experience is also very important for this application.

Simply Figuring Out the APIs and Data

This problem started when I was using the **OpenWeatherMap API** and the **OpenStreetMap** geocoding API, with this I struggled a lot with understanding of how the data was structured, especially with the nested for loops and the objects and arrays in some of the responses.

The way i went about trying to fix some of the errors i had with implementation was to firstly simply `console.log()` the data and explore the structure through there however i also used W3schools, Stack overflow and Web APIs MDN examples to get more comfortable with accessing pieces of data that i needed such as `data.current.weather`,

`data.current.temp;` and `data.current.wind_speed;`

Temperature Unit Toggle Button

At first when I implemented the simple toggle button for the temperature unit change it only changed the current weather temperature. Although this is a simple addition to my project it caused one of the most errors in not working and I had to fix it many times as the 5 day forecast would always struggle to update with the new unit added. To try and keep things more simple I kept the toggle working on only the current displayed temperature for now and

made a note that I needed a full toggle unit across all of the data. This was one of those times in my project where i had so many errors and could not find a fix however it was such a small issue in my app that i had to decide between a small trade or overcomplicating things and eating up my time for the deadline of this project so i decided to keep the user experience more smooth and bug free even if this did not fully work dynamically.

Evaluation

In my project looking back at it I do feel like I successfully brought together every simple step that I learnt in backend development, from using the API in general and handling the data to attempting to manage the user interaction and working with the **leaflet.js** . One of the biggest things that I think I learnt was how to work with APIs in general and getting used to reading different documentation online and figuring out how to fix things from sources and how to extract the right data that is needed. Although i know that this project is very simple and not the most outstanding backend development there are some things i would have liked to have hadd if i wanted a more detailed approach and had more time such as adding in a full support for my unit conversion, including the weather icons and wind data in the 5 day forecast and then maybe adding in loading indicators when the API is being processed, however overall i am proud of how this turned out seen as though i did struggle quite a bit with this backend development i have produced something i am proud of with a clean simple and still interactive and flowing weather and map application.